

Documentación Técnica Final — UTNotasApp

1. API y Servicios

1.1 Descripción de Endpoints

los endpoints marcados con  requieren cookie de sesión JWT válida (o header `Authorization: Bearer <token>` para clientes nativos). Los marcados con  requieren además rol ADMIN.

Query params: se indican en la columna *Request body* como `?param`.

Auth — `/api/auth`

Endpoint	Método	Descripción	Request body
<code>/api/auth/login</code>	POST	Autentica con email y contraseña. Devuelve cookie de sesión.	<code>email</code> : string, <code>password</code> : string, <code>captchaToken</code> : string (opcional)
<code>/api/auth/google</code>	POST	Autentica con token de Google OAuth. Devuelve cookie de sesión.	<code>credential</code> : string
<code>/api/auth/me</code>	GET 	Devuelve el perfil completo del usuario autenticado.	—
<code>/api/auth/logout</code>	POST 	Invalida la cookie de sesión.	—

Usuarios — `/api/users`

Endpoint	Método	Descripción	Request body
<code>/api/users/register</code>	POST	Registra un nuevo usuario.	<code>name</code> : string, <code>surname</code> : string, <code>email</code> : string, <code>username</code> : string, <code>password</code> : string, <code>careerId</code> : number
<code>/api/users/:id</code>	GET	Devuelve el perfil público de un usuario.	—
<code>/api/users/:id</code>	PATCH 	Actualiza datos del usuario. USER solo edita el suyo; ADMIN edita cualquiera. Todos los campos son opcionales.	<code>name</code> : string?, <code>surname</code> : string?, <code>email</code> : string?, <code>username</code> : string?, <code>password</code> : string?, <code>careerId</code> : number?, <code>profilePicture</code> : string? (URL o null)
<code>/api/users/:id</code>	DELETE 	Elimina el usuario. USER elimina solo el suyo;	—

Endpoint	Método	Descripción	Request body
ADMIN elimina cualquiera.			

Materiales — /api/materials

Endpoint	Método	Descripción	Request body
/api/materials	GET	Lista materiales con paginación por cursor.	?query: string?, ?tipo: string?, ?comision: string?, ?anioCursada: string?, ?userId: number?, ?limit: number?, ?cursor: number?
/api/materials/:id	GET	Devuelve el detalle de un material.	—
/api/materials	POST 	Crea un material. Requiere USER o ADMIN.	titulo: string, tipo: enum TipoMaterial, archivo: string (URL), descripcion: string? (null), comision: string? (null), añoCursada: number? (null), numeroParcial: number? (null), materiaId: number? (null), carreraId: number? (null)
/api/materials/:id	PATCH 	Edita un material propio. Requiere USER o ADMIN. Todos los campos son opcionales.	titulo: string?, tipo: enum TipoMaterial?, archivo: string? (URL), descripcion: string? (null), comision: string? (null), añoCursada: number? (null), numeroParcial: number? (null), materiaId: number? (null), carreraId: number? (null)
/api/materials/:id	DELETE 	Elimina un material propio. Requiere USER o ADMIN.	—

Enum TipoMaterial: PARCIAL, PARCIAL_RESUELTO, FINAL, FINAL_RESUELTO, PRACTICA, PRACTICA_RESUELTA, APUNTE, RESUMEN, OTRO

Calificaciones (votos) — /api/materials/:materialId/calificaciones

Endpoint	Método	Descripción	Request body
/api/materials/:materialId/calificaciones	GET 	Lista votos del material. Filtrable por userId.	?userId: number?
/api/materials/:materialId/calificaciones/me	GET 	Devuelve el voto propio del	—

Endpoint	Método	Descripción	Request body
		usuario autenticado sobre el material.	
/api/materials/:materialId/calificaciones/me	POST 	Crea el voto del usuario autenticado sobre el material.	materialId: number, value: boolean (true = upvote, false = downvote)
/api/materials/:materialId/calificaciones/me	PATCH 	Actualiza el voto existente del usuario autenticado.	value: boolean
/api/materials/:materialId/calificaciones/me	DELETE 	Elimina el voto del usuario autenticado sobre el material.	—

Favoritos — /api/favoritos

Endpoint	Método	Descripción	Request body
/api/favoritos	GET 	Lista los materiales marcados como favoritos por el usuario autenticado.	—
/api/favoritos/:materialId	GET 	Devuelve el favorito propio del usuario sobre el material (o 404 si no existe).	—
/api/favoritos/:materialId	POST 	Alterna el estado de favorito sobre el material (toggle).	—

Reportes — /api/reportes

Endpoint	Método	Descripción	Request body
/api/reportes	GET 	Lista todos los reportes del sistema (solo ADMIN).	—

Endpoint	Método	Descripción	Request body
/api/reportes/:materialId/me	GET 	Devuelve el reporte propio del usuario autenticado sobre el material.	—
/api/reportes/:materialId/me	POST 	Crea un reporte del usuario autenticado sobre el material.	motivo : enum MotivoReporte, descripcion : string? (null, máx 500 chars)
/api/reportes/:id	DELETE 	Elimina un reporte por su id (solo ADMIN).	—

Enum MotivoReporte: CONTENIDO_INAPROPIADO, SPAM, PLAGIO, OTRO

Carreras — /api/carreras

Endpoint	Método	Descripción	Request body
/api/carreras	GET	Lista todas las carreras.	—
/api/carreras/:id	GET	Detalle de una carrera con sus materias asociadas.	—
/api/carreras/materia/:materialId	GET	Lista las carreras que dictan una materia dada.	—
/api/carreras/:id/materias/:materialId	GET	Consulta la relación carrera-materia (año en que se cursa).	—
/api/carreras	POST 	Crea una carrera (solo ADMIN). Puede incluir materias anidadas.	nombre : string, icon : string, materias : { materialId : number, anio : number }[]?
/api/carreras/:id	PATCH 	Actualiza datos de una carrera (solo ADMIN).	nombre : string?, icon : string?
/api/carreras/:id	DELETE 	Elimina una carrera (solo ADMIN).	—
/api/carreras/:id/materias	POST 	Asigna o actualiza una materia dentro	materialId : number, anio : number (1–7)

Endpoint	Método	Descripción	Request body
		de la carrera (upsert).	
<code>/api/carreras/:id/materias/:materiaId</code>	DELETE 	Desasigna una materia de la carrera (solo ADMIN).	—

Materias — `/api/materias`

Endpoint	Método	Descripción	Request body
<code>/api/materias</code>	GET	Lista todas las materias (paginado).	—
<code>/api/materias/:id</code>	GET	Detalle de una materia.	—
<code>/api/materias</code>	POST 	Crea una materia (solo ADMIN).	<code>nombre</code> : string, <code>descripcion</code> : string
<code>/api/materias/:id</code>	PATCH 	Actualiza una materia (solo ADMIN).	<code>nombre</code> : string?, <code>descripcion</code> : string?

1.2 Repositorio de Archivos

Los archivos subidos (PDFs, imágenes, documentos) **no se almacenan en el servidor propio**. El campo `archivo` en la entidad `Material` es un **string URL** que apunta a un recurso externo ya alojado. El flujo es:

1. El cliente selecciona un archivo local (via `expo-document-picker` o cámara via `expo-image-picker`).
2. El archivo se sube a un servicio de almacenamiento externo (ej. Cloudinary) desde un endpoint dedicado. (aún no implementado debido a que el software es muy limitado para su uso gratuito y muy costoso para su uso pago)
3. La URL resultante se envía al backend en el campo `archivo` del `POST /api/materials`.
4. El backend almacena únicamente la URL en la columna `archivo` (VARCHAR) de la tabla `materials` en PostgreSQL.

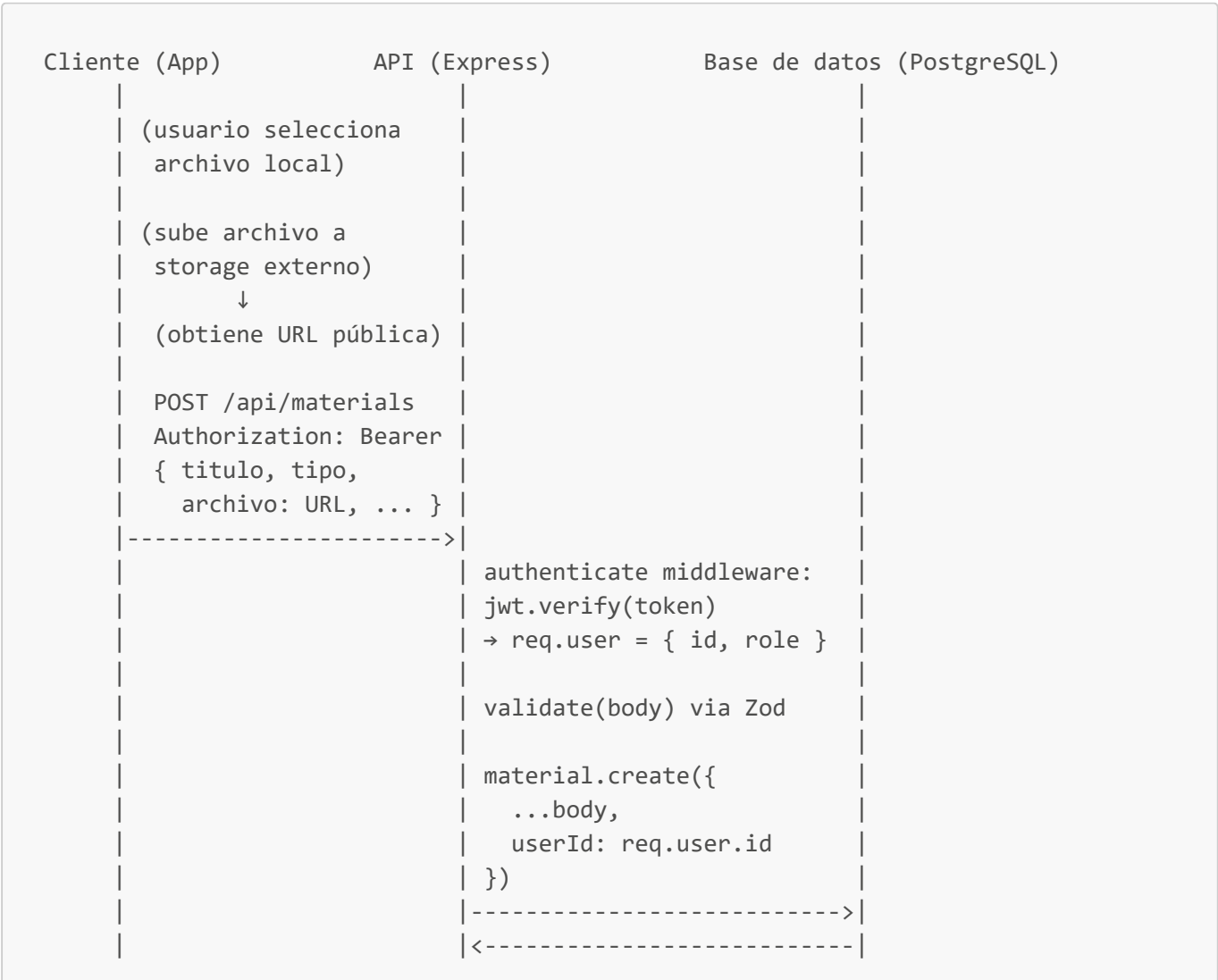
1.3 Diagramas de Secuencia

Flujo 1: Login con email y contraseña



```
| |----->|
| |-----<|
| (user row o null)
|
| bcrypt.compare(
|   password,
|   user?.hash ?? DUMMY_HASH
| ) ← timing attack safe
|
| jwt.sign({ id, email,
|   role }, JWT_SECRET, 7d)
|
| 200 { data: { user,
|   token } }
| Set-Cookie: token=...
|-----<|
| (cliente nativo guarda
|   token en memoria;
|   web usa cookie)
| |
```

Flujo 2: Subida de material



```

| | (material row creado) |
| | |
| 201 { data: material } |
| <----- |
| |
| (React Query invalida |
| cache ["materials"]) |

```

2. Seguridad y Manejo de Datos

2.1 Datos del usuario almacenados y cómo están protegidos

La tabla **users** almacena los siguientes campos:

Campo	Tipo	Protección
id	INT (PK autoincrement)	Solo interno, nunca expuesto en URLs de usuario
email	VARCHAR UNIQUE	Nunca devuelto en respuestas públicas de perfil
username	VARCHAR UNIQUE	Público
name / surname	VARCHAR(50)	Público en perfil
password	VARCHAR	Almacenado como hash bcrypt (cost factor 10). Nunca serializado en respuestas — se desestructura y descarta explícitamente en auth.service.ts antes de retornar
careerId	INT (FK)	Público
role	ENUM (USER / ADMIN)	Solo se lee del JWT, nunca aceptado en body
profilePicture	VARCHAR(1000)?	URL externa, opcional
points	INT	Público en perfil
createdAt / updatedAt	TIMESTAMP	Solo accesible vía perfil propio

Medidas adicionales:

- El campo **password** usa **bcrypt** con un hash estático de fallback (**DUMMY_HASH**) para equiparar el tiempo de respuesta cuando el email no existe, neutralizando ataques de timing.
- Los endpoints de mutación sobre usuarios aplican **ABAC** (control de acceso basado en atributos): un **USER** solo puede modificar o eliminar su propio registro; un **ADMIN** puede operar sobre cualquiera.
- El mensaje de error en login es genérico ("credenciales no válidas") para no revelar si el email está registrado o no.

2.2 Política de sesiones y tokens

Aspecto	Comportamiento
Mecanismo	JWT firmado con <code>JWT_SECRET</code> (HS256 por defecto de <code>jsonwebtoken</code>)
Expiración	Configurable via <code>JWT_EXPIRES_IN</code> en variables de entorno. Valor por defecto: 7 días
Transporte (web)	Cookie <code>httpOnly, sameSite: lax, secure: true</code> en producción (HTTPS). Inaccesible desde JavaScript del cliente → inmune a XSS
Transporte (nativo)	El token también se devuelve en el body (<code>data.token</code>). El cliente React Native lo guarda en memoria y lo envía como <code>Authorization: Bearer <token></code>
Renovación	No hay refresh token. Al expirar el JWT, el usuario debe volver a autenticarse. El middleware detecta <code>TokenExpiredError</code> y responde 401
Revocación	El logout limpia la cookie del lado del servidor con <code>res.clearCookie</code> . No hay blacklist de tokens; la invalidación depende de la expiración natural del JWT
Validación de forma	El middleware <code>authenticate</code> valida la estructura JWT (3 segmentos base64url) con una regex antes de llamar a <code>jwt.verify</code> , evitando excepciones innecesarias

2.3 Validaciones de entrada

Client-side (React Native / Expo)

Las validaciones del formulario se ejecutan en el hook `useMaterialForm` y la función `validateForm` antes de llamar a la API:

- **Título:** requerido, no vacío.
- **Tipo de material:** requerido, debe ser uno de los valores del enum `TipoMaterial`.
- **Archivos:** al menos un archivo adjunto requerido para crear un material.
- **Año de cursada:** solo dígitos, máximo 4 caracteres (`/\D/g` strip en `onChangeText`).
- **Comisión:** máximo 10 caracteres, convertido a mayúsculas automáticamente.
- **Materia / Carrera:** selección desde catálogo (no entrada libre), validada contra la API de carreras y materias.

Los errores se muestran inline bajo cada campo usando el componente `FormField` con su prop `error`.

Server-side (Express + Zod)

Todos los endpoints con body pasan por el middleware `validate(schema)` que aplica un schema Zod estricto (`.strict()` rechaza campos no declarados). Los contratos por dominio son:

Dominio	Constraint notable
Auth	<code>email</code> validado con <code>z.string().email()</code> . <code>password</code> min 1 char (complejidad delegada al registro)
Registro	<code>password</code> min 8 chars, al menos una mayúscula y un número. <code>email</code> normalizado a lowercase

Dominio	Constraint notable
Material	<code>tipo</code> validado contra el enum Prisma <code>TipoMaterial</code> . <code>archivo</code> min 1 char. <code>añoCursada</code> entero positivo $\leq$ año actual
Reporte	<code>motivo</code> validado contra el enum Prisma <code>MotivoReporte</code> . <code>descripcion</code> máx 500 chars
Carrera/Materia	<code>año</code> entre 1 y 7 (años de carrera universitaria). <code>nombre</code> e <code>icon</code> max 100 chars

Los errores de validación Zod son mapeados por el middleware `handleError` a respuestas 400 con detalle de campos inválidos.

CORS: la API aplica una lista blanca de orígenes (`FRONTEND_URL` en env + `localhost:8081` para desarrollo). Requests del APK llegan sin `Origin` header (`undefined`) y son permitidos explícitamente.

3. Performance y Optimizaciones

3.1 Medición del tiempo de inicio de la app

[A completar por el equipo con mediciones reales en dispositivo]

3.2 Uso de memoria y consumo de batería

[A completar por el equipo con mediciones reales en dispositivo]

3.3 Estrategias de caché y lazy loading

Caché de datos — React Query

La app usa **TanStack React Query v5** como capa de caché sobre la API. El `QueryClient` se inicializa una sola vez en el layout raíz y persiste durante toda la sesión.

Query	<code>staleTime</code>	Razonamiento
Lista de materias (<code>/api/materias</code>)	10 minutos	El catálogo académico cambia poco; no vale re-pedir en cada navegación
Lista de carreras (<code>/api/carreras</code>)	10 minutos	Mismo razonamiento que materias
Carreras por materia	10 minutos	Derivado del catálogo
Detalle de material individual	5 minutos	Puede cambiar (votos, ediciones) pero no en tiempo real
Lista de materiales (búsqueda)	2 minutos	Más dinámica: nuevas subidas deben aparecer relativamente rápido
Voto / favorito / reporte propios	Sin <code>staleTime</code> (default 0)	Estado personal, siempre fresco al enfocar la pantalla

Query	staleTime	Razonamiento
Reportes admin	30 segundos	Panel que un admin puede refrescar manualmente

Tras una mutación exitosa, React Query **invalida las queries afectadas** (`queryClient.invalidateQueries`) o actualiza el cache directamente (`queryClient.setQueryData`) para evitar un round-trip innecesario.

Paginación por cursor

El endpoint `GET /api/materials` usa paginación por cursor (`?cursor=<lastId>&limit=N`) en lugar de offset, lo que evita escaneos de tabla completos en PostgreSQL a medida que crece el dataset.

Lazy loading de pantallas

Expo Router usa **lazy loading automático** de segmentos de ruta: las pantallas solo se cargan (bundle JS evaluado) cuando el usuario navega hacia ellas por primera vez. Las pestañas del bottom tab navigator que no se han visitado no se montan hasta que el usuario las abre.

Imágenes

El componente `expo-image` (usado para avatares y previews) soporta caché de disco automático, evitando re-descargas de la misma URL entre sesiones.